In [4]:
```
import pandas as pd
import numpy as np
import os
print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

In [7]:
```
df = pd.read_csv("netflix.csv")
print("Dataset loaded successfully!")
print("Number of rows:", df.shape[0])
print("Number of columns:", df.shape[1])
df.head()
```

```
Dataset loaded successfully!
Number of rows: 8790
Number of columns: 10
```

Out[7]:

	show_id	type	title	director	country	date_added	release_year	rating	duration
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	United States	9/25/2021	2020	PG-13	90 min
1	s3	TV Show	Ganglands	Julien Leclercq	France	9/24/2021	2021	TV-MA	1 Season
2	s6	TV Show	Midnight Mass	Mike Flanagan	United States	9/24/2021	2021	TV-MA	1 Season
3	s14	Movie	Confessions of an Invisible Girl	Bruno Garotti	Brazil	9/22/2021	2021	TV-PG	91 min
4	s8	Movie	Sankofa	Haile Gerima	United States	9/24/2021	1993	TV-MA	125 min

In [8]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8790 entries, 0 to 8789
Data columns (total 10 columns):
 #   Column        Non-Null Count  Dtype 
---  ------        --------------  ----- 
 0   show_id       8790 non-null   object
 1   type          8790 non-null   object
 2   title         8790 non-null   object
 3   director      8790 non-null   object
 4   country       8790 non-null   object
 5   date_added    8790 non-null   object
 6   release_year  8790 non-null   int64 
 7   rating        8790 non-null   object
 8   duration      8790 non-null   object
 9   listed_in     8790 non-null   object
dtypes: int64(1), object(9)
memory usage: 686.8+ KB
```

In [9]:
```
df.describe(include="all")
```

Out[9]:

	show_id	type	title	director	country	date_added	release_year	rating	duration
count	8790	8790	8790	8790	8790	8790	8790.000000	8790	8790
unique	8790	2	8787	4528	86	1713	NaN	14	220
top	s8786	Movie	9-Feb	Not Given	United States	1/1/2020	NaN	TV-MA	1 Season
freq	1	6126	2	2588	3240	110	NaN	3205	1791
mean	NaN	NaN	NaN	NaN	NaN	NaN	2014.183163	NaN	NaN
std	NaN	NaN	NaN	NaN	NaN	NaN	8.825466	NaN	NaN
min	NaN	NaN	NaN	NaN	NaN	NaN	1925.000000	NaN	NaN
25%	NaN	NaN	NaN	NaN	NaN	NaN	2013.000000	NaN	NaN
50%	NaN	NaN	NaN	NaN	NaN	NaN	2017.000000	NaN	NaN
75%	NaN	NaN	NaN	NaN	NaN	NaN	2019.000000	NaN	NaN
max	NaN	NaN	NaN	NaN	NaN	NaN	2021.000000	NaN	NaN

In [10…
```
print("Missing values in each column:")
print(df.isnull().sum())
```

```
Missing values in each column:
show_id         0
type            0
title           0
director        0
country         0
date_added      0
release_year    0
rating          0
duration        0
listed_in       0
dtype: int64
```

In [11…
```python
print("Duplicate rows:", df.duplicated().sum())
```

```
Duplicate rows: 0
```

In [12…
```python
df = df.drop_duplicates()
print("Duplicate rows after cleaning:", df.duplicated().sum())
```

```
Duplicate rows after cleaning: 0
```

In [13…
```python
# Fill missing text values
text_columns = [
    "director",
    "country",
    "rating",
    "duration",
    "listed_in"
]

for col in text_columns:
    df[col] = df[col].fillna("Unknown")

# Convert date_added into a date format
df["date_added"] = pd.to_datetime(
    df["date_added"],
    errors="coerce"
)

print("Missing values after cleaning:")
print(df.isnull().sum())
```

```
Missing values after cleaning:
show_id         0
type            0
title           0
director        0
country         0
date_added      0
release_year    0
rating          0
duration        0
listed_in       0
dtype: int64
```

In [14…
```
for col in [
    "type",
    "title",
    "director",
    "country",
    "rating",
    "duration",
    "listed_in"
]:
    df[col] = df[col].astype("string").str.strip()

print("Text formatting completed successfully!")
```

```
Text formatting completed successfully!
```

In [15…
```
os.makedirs("Output", exist_ok=True)
```

In [16…
```
df.to_csv("Output/netflix_cleaned.csv", index=False)
print("Cleaned dataset saved successfully!")
```

```
Cleaned dataset saved successfully!
```

In [17…
```
# Check the cleaned dataset
print("Dataset shape:", df.shape)
print("\nMissing values:")
print(df.isnull().sum())
print("\nDuplicate rows:")
print(df.duplicated().sum())
print("\nFirst five records:")
display(df.head())
```

```
Dataset shape: (8790, 10)

Missing values:
show_id         0
type            0
title           0
director        0
country         0
date_added      0
release_year    0
rating          0
duration        0
listed_in       0
dtype: int64

Duplicate rows:
0

First five records:
```

	show_id	type	title	director	country	date_added	release_year	rating	duration
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	United States	2021-09-25	2020	PG-13	90 min
1	s3	TV Show	Ganglands	Julien Leclercq	France	2021-09-24	2021	TV-MA	1 Season
2	s6	TV Show	Midnight Mass	Mike Flanagan	United States	2021-09-24	2021	TV-MA	1 Season
3	s14	Movie	Confessions of an Invisible Girl	Bruno Garotti	Brazil	2021-09-22	2021	TV-PG	91 min
4	s8	Movie	Sankofa	Haile Gerima	United States	2021-09-24	1993	TV-MA	125 min

In [18…

```
df.to_csv("Output/netflix_cleaned.csv", index=False)
print("Final cleaned dataset saved successfully!")
```

```
Final cleaned dataset saved successfully!
```

In [1]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import os
print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

In [2]:
```
df = pd.read_csv("Output/netflix_cleaned.csv")
print("Dataset loaded successfully!")
print("Rows:", df.shape[0])
print("Columns:", df.shape[1])
display(df.head())
```

```
Dataset loaded successfully!
Rows: 8790
Columns: 10
```

	show_id	type	title	director	country	date_added	release_year	rating	duration
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	United States	2021-09-25	2020	PG-13	90 min
1	s3	TV Show	Ganglands	Julien Leclercq	France	2021-09-24	2021	TV-MA	1 Season
2	s6	TV Show	Midnight Mass	Mike Flanagan	United States	2021-09-24	2021	TV-MA	1 Season
3	s14	Movie	Confessions of an Invisible Girl	Bruno Garotti	Brazil	2021-09-22	2021	TV-PG	91 min
4	s8	Movie	Sankofa	Haile Gerima	United States	2021-09-24	1993	TV-MA	125 min

In [4]:
```
print("Dataset Information:")
df.info()
```

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8790 entries, 0 to 8789
Data columns (total 10 columns):
 #   Column        Non-Null Count  Dtype 
---  ------        --------------  ----- 
 0   show_id       8790 non-null   object
 1   type          8790 non-null   object
 2   title         8790 non-null   object
 3   director      8790 non-null   object
 4   country       8790 non-null   object
 5   date_added    8790 non-null   object
 6   release_year  8790 non-null   int64 
 7   rating        8790 non-null   object
 8   duration      8790 non-null   object
 9   listed_in     8790 non-null   object
dtypes: int64(1), object(9)
memory usage: 686.8+ KB
```

In [5]:
```
print("Statistical Summary:")
display(df.describe(include="all"))
```

Statistical Summary:

	show_id	type	title	director	country	date_added	release_year	rating	du
count	8790	8790	8790	8790	8790	8790	8790.000000	8790	
unique	8790	2	8786	4528	86	1713	NaN	14	
top	s8786	Movie	Consequences	Not Given	United States	2020-01-01	NaN	TV-MA	1 S
freq	1	6126	2	2588	3240	110	NaN	3205	
mean	NaN	NaN	NaN	NaN	NaN	NaN	2014.183163	NaN	
std	NaN	NaN	NaN	NaN	NaN	NaN	8.825466	NaN	
min	NaN	NaN	NaN	NaN	NaN	NaN	1925.000000	NaN	
25%	NaN	NaN	NaN	NaN	NaN	NaN	2013.000000	NaN	
50%	NaN	NaN	NaN	NaN	NaN	NaN	2017.000000	NaN	
75%	NaN	NaN	NaN	NaN	NaN	NaN	2019.000000	NaN	
max	NaN	NaN	NaN	NaN	NaN	NaN	2021.000000	NaN	

In [6]:
```
content_counts = df["type"].value_counts()

print("Content Distribution:")
print(content_counts)
```

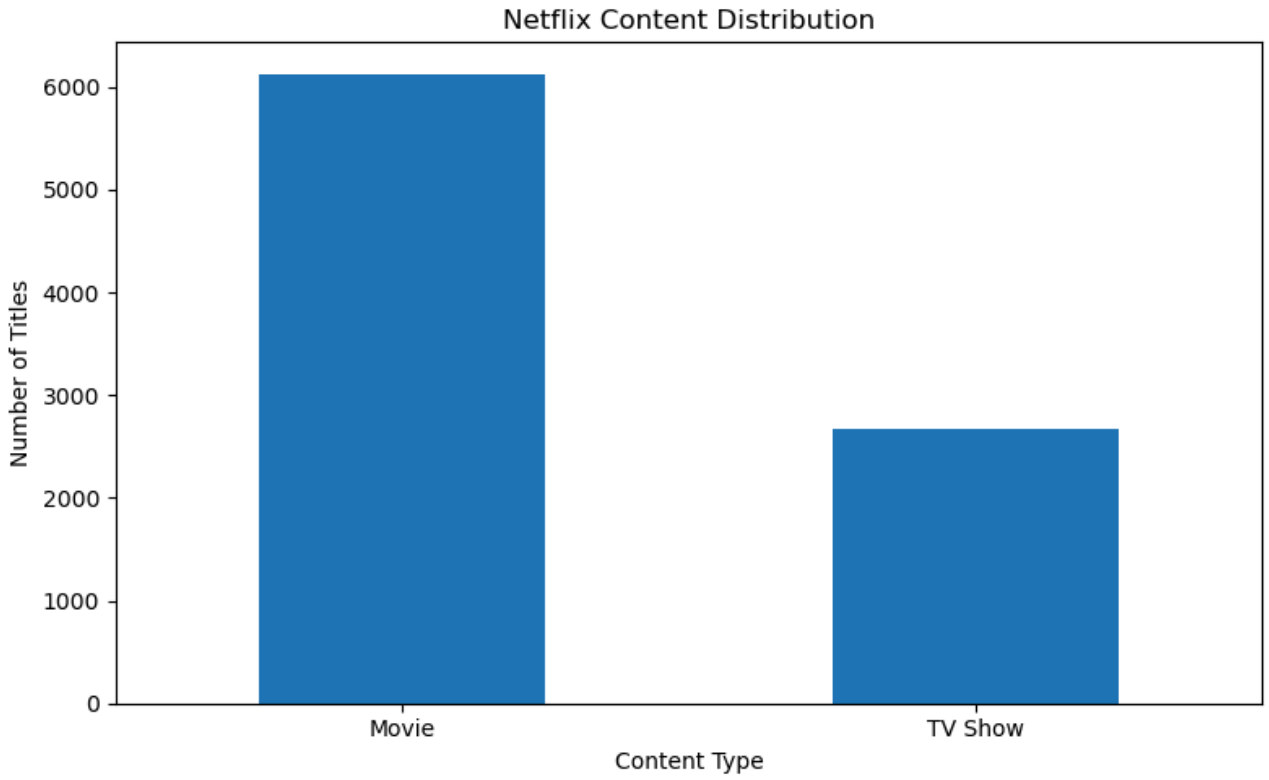
```
Content Distribution:
type
Movie      6126
TV Show    2664
Name: count, dtype: int64
```

In [7]:
```
plt.figure(figsize=(8, 5))

content_counts.plot(kind="bar")

plt.title("Netflix Content Distribution")
plt.xlabel("Content Type")
plt.ylabel("Number of Titles")

plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```



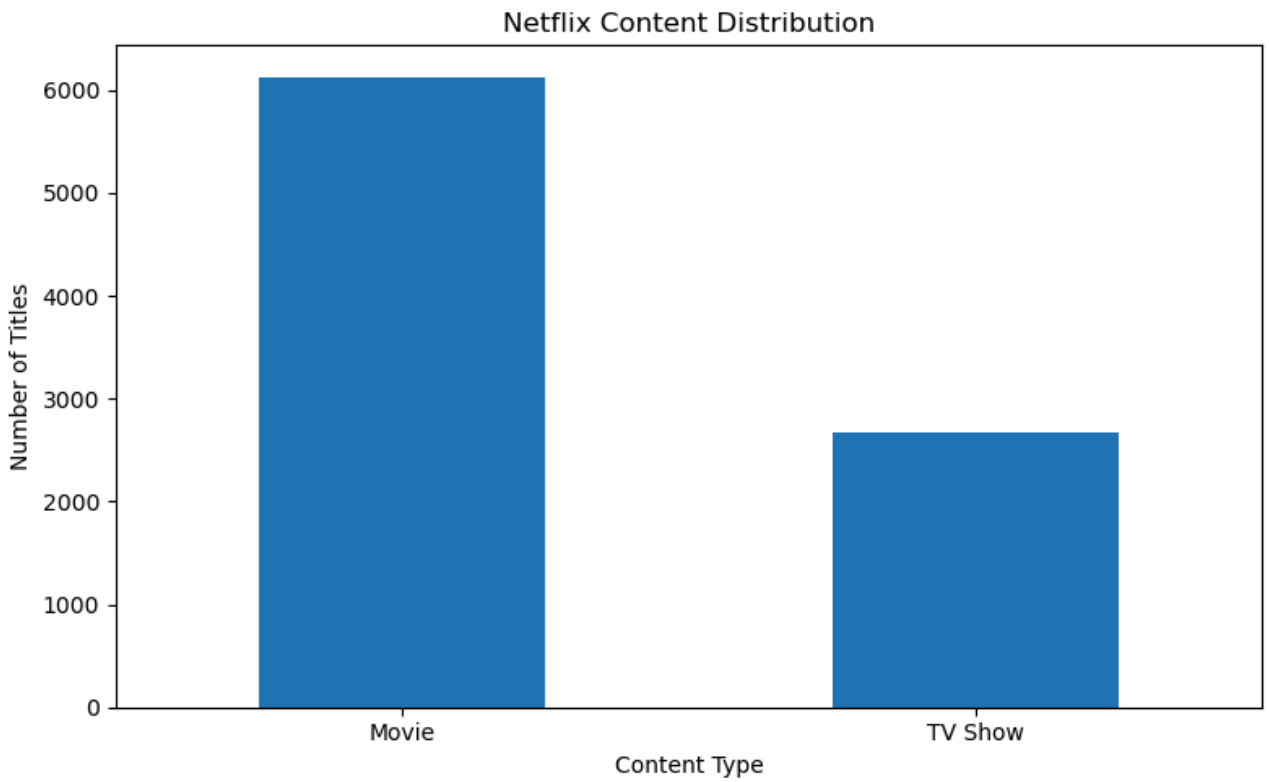
In [8]:
```
os.makedirs("Graphs", exist_ok=True)
plt.figure(figsize=(8, 5))
content_counts.plot(kind="bar")

plt.title("Netflix Content Distribution")
plt.xlabel("Content Type")
plt.ylabel("Number of Titles")

plt.xticks(rotation=0)
plt.tight_layout()

plt.savefig(
    "Graphs/content_distribution.png",
    bbox_inches="tight"
)
plt.show()
```

```
print("Graph saved successfully!")
```




```
Graph saved successfully!
```

In [9]:
```
# Analyze Netflix content by country

countries = df["country"].dropna().str.split(", ").explode()

# Remove unknown countries
countries = countries[countries != "Unknown"]

# Find the top 10 countries
top_countries = countries.value_counts().head(10)

print("Top 10 Countries with the Most Netflix Content:")
print(top_countries)
```

```
Top 10 Countries with the Most Netflix Content:
country
United States     3240
India             1057
United Kingdom     638
Pakistan           421
Not Given          287
Canada             271
Japan              259
South Korea        214
France             213
Spain              182
Name: count, dtype: int64
```

In [10…

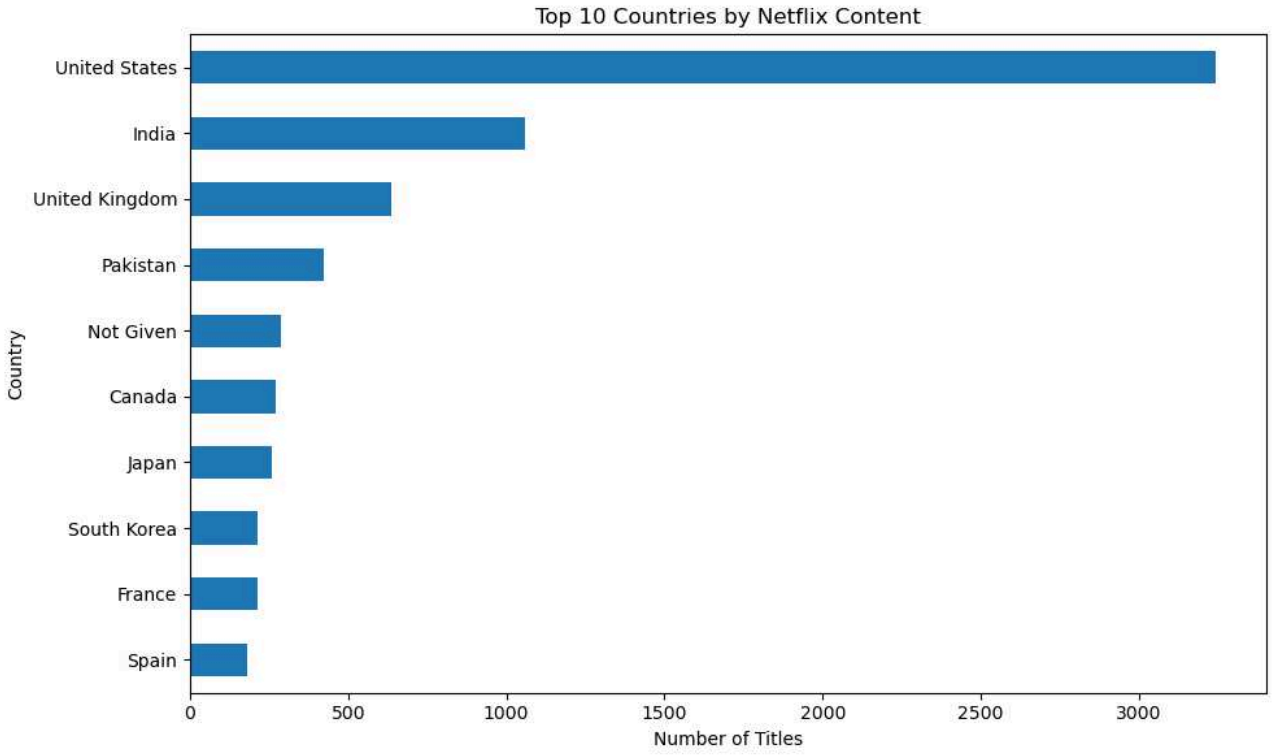
```
plt.figure(figsize=(10, 6))

top_countries.sort_values().plot(kind="barh")

plt.title("Top 10 Countries by Netflix Content")
plt.xlabel("Number of Titles")
plt.ylabel("Country")

plt.tight_layout()
plt.savefig("Graphs/top_countries.png", bbox_inches="tight")
plt.show()

print("Country graph saved successfully!")
```




```
Country graph saved successfully!
```

In [11…

```
# Analyze Netflix content by genre

genres = df["listed_in"].dropna().str.split(", ").explode()

top_genres = genres.value_counts().head(10)

print("Top 10 Netflix Genres:")
print(top_genres)
```

```
Top 10 Netflix Genres:
listed_in
International Movies        2752
Dramas                      2426
Comedies                    1674
International TV Shows      1349
Documentaries                869
Action & Adventure           859
TV Dramas                    762
Independent Movies           756
Children & Family Movies     641
Romantic Movies              616
Name: count, dtype: int64
```

In [12…

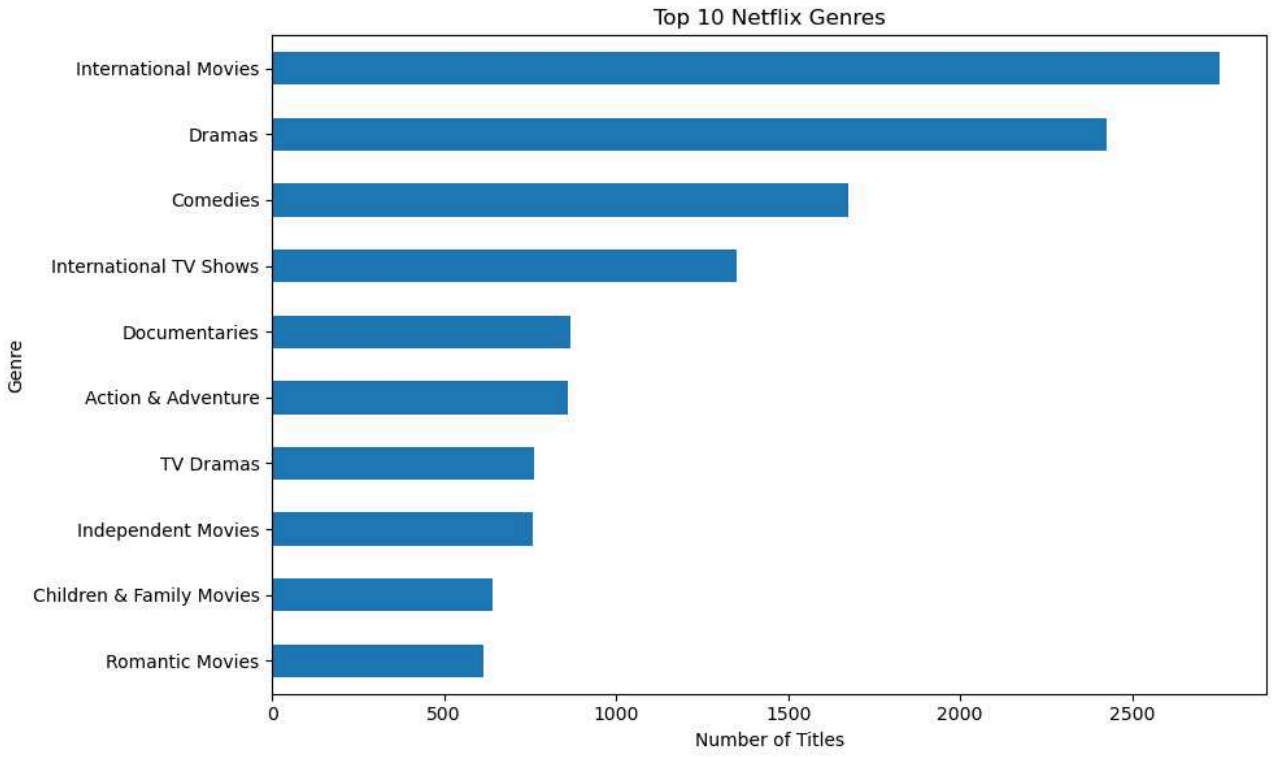
```python
plt.figure(figsize=(10, 6))

top_genres.sort_values().plot(kind="barh")

plt.title("Top 10 Netflix Genres")
plt.xlabel("Number of Titles")
plt.ylabel("Genre")

plt.tight_layout()
plt.savefig("Graphs/top_genres.png", bbox_inches="tight")
plt.show()

print("Genre graph saved successfully!")
```




```
Genre graph saved successfully!
```

In [13…

```python
# Convert release_year to numeric
```

```
df["release_year"] = pd.to_numeric(
    df["release_year"],
    errors="coerce"
)

# Count titles released each year
yearly_counts = df["release_year"].dropna().value_counts().sort_index()

print("Yearly content analysis:")
print(yearly_counts.tail(15))
```

```
Yearly content analysis:
release_year
2007      88
2008     135
2009     152
2010     192
2011     185
2012     236
2013     286
2014     352
2015     555
2016     901
2017    1030
2018    1146
2019    1030
2020     953
2021     592
Name: count, dtype: int64
```

In [14…

```
plt.figure(figsize=(12, 6))

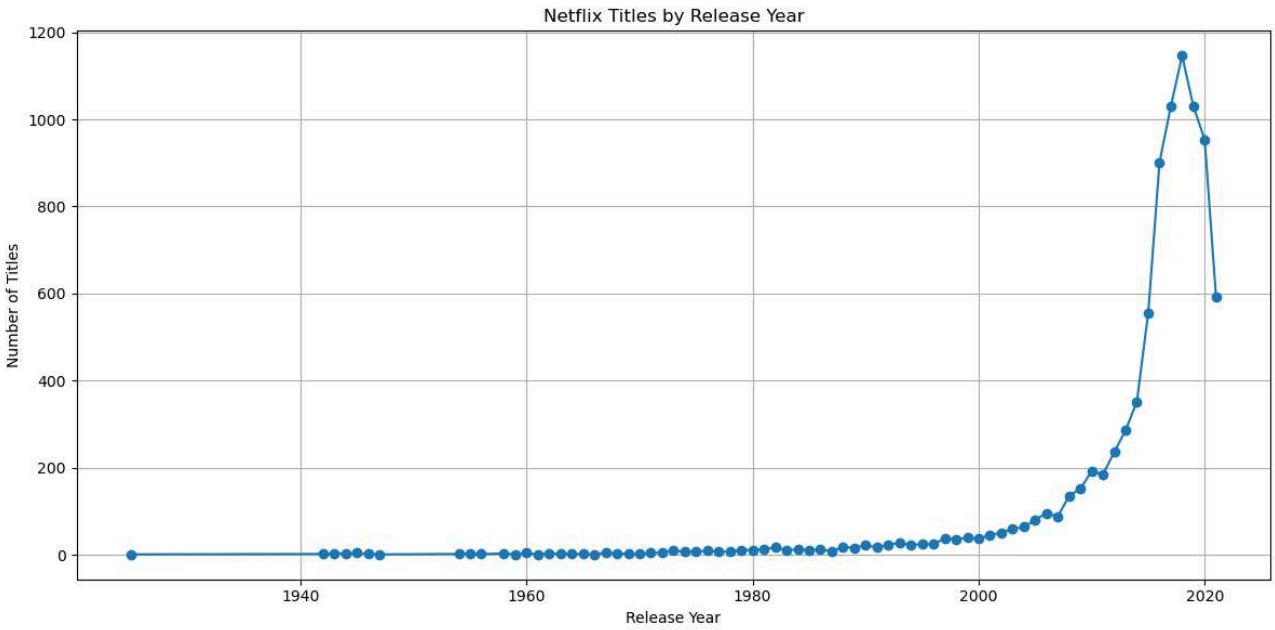
plt.plot(
    yearly_counts.index,
    yearly_counts.values,
    marker="o"
)

plt.title("Netflix Titles by Release Year")
plt.xlabel("Release Year")
plt.ylabel("Number of Titles")

plt.grid(True)
plt.tight_layout()

plt.savefig("Graphs/yearly_trends.png", bbox_inches="tight")
plt.show()

print("Yearly trend graph saved successfully!")
```




```
Yearly trend graph saved successfully!
```

Exploratory Data Analysis — Conclusion

Objectives

- Analyze the distribution of Movies and TV Shows.
- Identify the countries contributing the most content.
- Explore the most common genres.
- Study the distribution of content by release year.

Key Findings

1. The dataset contains more Movies than TV Shows.
2. The top 10 countries have been identified using country-level counts.
3. The top 10 genres have been identified using genre-level counts.
4. The release-year visualization shows how the content distribution varies across years.

Conclusion

Exploratory Data Analysis helped identify patterns in the Netflix dataset. Bar charts and line charts were used to summarize content types, countries,

genres, and release years. These findings provide a foundation for building a recommendation system and analyzing content trends.

In [1]:
```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

In [2]:
```
df = pd.read_csv("Output/netflix_cleaned.csv")
print("Dataset loaded successfully!")
print("Rows:", df.shape[0])
print("Columns:", df.shape[1])
df.head()
```

```
Dataset loaded successfully!
Rows: 8790
Columns: 10
```

Out[2]:

	show_id	type	title	director	country	date_added	release_year	rating	duration
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	United States	2021-09-25	2020	PG-13	90 min
1	s3	TV Show	Ganglands	Julien Leclercq	France	2021-09-24	2021	TV-MA	1 Season
2	s6	TV Show	Midnight Mass	Mike Flanagan	United States	2021-09-24	2021	TV-MA	1 Season
3	s14	Movie	Confessions of an Invisible Girl	Bruno Garotti	Brazil	2021-09-22	2021	TV-PG	91 min
4	s8	Movie	Sankofa	Haile Gerima	United States	2021-09-24	1993	TV-MA	125 min

In [4]:
```
features = [
    "type",
    "director",
    "country",
    "rating",
    "listed_in"
]

for col in features:
    df[col] = df[col].fillna("Unknown").astype(str)

df["combined_features"] = df[features].agg(
    " ".join,
```

```
    axis=1
)
display(df[["title", "combined_features"]].head())
```

	title	combined_features
0	Dick Johnson Is Dead	Movie Kirsten Johnson United States PG-13 Docu...
1	Ganglands	TV Show Julien Leclercq France TV-MA Crime TV ...
2	Midnight Mass	TV Show Mike Flanagan United States TV-MA TV D...
3	Confessions of an Invisible Girl	Movie Bruno Garotti Brazil TV-PG Children & Fa...
4	Sankofa	Movie Haile Gerima United States TV-MA Dramas,...

In [5]:
```
tfidf = TfidfVectorizer(stop_words="english")

tfidf_matrix = tfidf.fit_transform(
    df["combined_features"]
)
print("TF-IDF conversion completed!")
print("Matrix shape:", tfidf_matrix.shape)
```

```
TF-IDF conversion completed!
Matrix shape: (8790, 6569)
```

In [6]:
```
similarity_matrix = cosine_similarity(
    tfidf_matrix,
    dense_output=False
)
print("Similarity calculation completed!")
```

```
Similarity calculation completed!
```

In [7]:
```
def recommend_titles(title, top_n=5):
    matches = df.index[
        df["title"].str.casefold() == title.casefold()
    ].tolist()

    if not matches:
        print("Title not found in the dataset.")
        return
    idx = matches[0]
    scores = similarity_matrix[idx].toarray().ravel()

    # Exclude the selected title itself
    scores[idx] = -1

    similar_indices = scores.argsort()[::-1][:top_n]

    recommendations = df.iloc[similar_indices][
        ["title", "type", "listed_in", "release_year"]
    ].copy()

    recommendations["similarity_score"] = scores[similar_indices]
```

```
    return recommendations
```

In [8]:
```
recommend_titles("Ganglands", 5)
```

Out[8]:

	title	type	listed_in	release_year	similarity_score
1183	Sentinelle	Movie	Action & Adventure, Dramas, International Movies	2021	0.847388
2142	Earth and Blood	Movie	Dramas, International Movies, Thrillers	2020	0.773519
8408	Crime Time	TV Show	Crime TV Shows, International TV Shows, TV Act...	2017	0.684723
6738	Lupin	TV Show	Crime TV Shows, International TV Shows, TV Act...	2021	0.684723
8065	La Mante	TV Show	Crime TV Shows, International TV Shows, TV Dramas	2017	0.616748

In [9]:
```
df["title"].head(20).tolist()
```

Out[9]:
```
['Dick Johnson Is Dead',
 'Ganglands',
 'Midnight Mass',
 'Confessions of an Invisible Girl',
 'Sankofa',
 'The Great British Baking Show',
 'The Starling',
 'Motu Patlu in the Game of Zones',
 'Je Suis Karl',
 'Motu Patlu in Wonderland',
 'Motu Patlu: Deep Sea Adventure',
 'Motu Patlu: Mission Moon',
 '99 Songs (Tamil)',
 'Bridgerton - The Afterparty',
 'Bling Empire - The Afterparty',
 'Cobra Kai - The Afterparty',
 'The Circle - The Afterparty',
 'Jailbirds New Orleans',
 'Crime Stories: India Detectives',
 'True: Winter Wishes']
```

In [10…
```
recommend_titles("The Crown", 5)
```

Out[10]:

	title	type	listed_in	release_year	similarity_score
7799	Black Earth Rising	TV Show	British TV Shows, International TV Shows, TV D...	2018	1.0
7745	Traitors	TV Show	British TV Shows, International TV Shows, TV D...	2019	1.0
8026	Requiem	TV Show	British TV Shows, International TV Shows, TV D...	2018	1.0
7677	Black Mirror	TV Show	British TV Shows, International TV Shows, TV D...	2019	1.0
6867	Behind Her Eyes	TV Show	British TV Shows, International TV Shows, TV D...	2021	1.0

In [11…

```
recommend_titles("Breaking Bad", 5)
```

Out[11]:

	title	type	listed_in	release_year	similarity_score
7674	Designated Survivor	TV Show	Crime TV Shows, TV Dramas, TV Thrillers	2019	1.000000
7309	Ozark	TV Show	Crime TV Shows, TV Dramas, TV Thrillers	2020	1.000000
6724	The Assassination of Gianni Versace	TV Show	Crime TV Shows, TV Dramas, TV Thrillers	2018	1.000000
6925	Dare Me	TV Show	Crime TV Shows, TV Dramas, TV Thrillers	2019	1.000000
7078	The Blacklist	TV Show	Crime TV Shows, TV Dramas, TV Thrillers	2019	0.947209

Recommendation System — Conclusion

Objective

To develop a content-based recommendation system for Netflix titles.

Methodology

1. Selected content features such as type, director, country, rating, and genre.
2. Combined the features into a single text representation.
3. Used TF-IDF to convert text into numerical vectors.
4. Calculated cosine similarity between titles.
5. Generated recommendations for a selected title.

Conclusion

The recommendation system identifies Netflix titles with similar metadata. It demonstrates how text processing and similarity measures can be used to build a basic content-based recommendation system.

In [ ]:

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score
print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

In [3]:
```
df = pd.read_csv("Output/netflix_cleaned.csv")
print("Dataset loaded successfully!")
print("Rows:", df.shape[0])
print("Columns:", df.shape[1])
df.head()
```

```
Dataset loaded successfully!
Rows: 8790
Columns: 10
```

Out[3]:

	show_id	type	title	director	country	date_added	release_year	rating	duration
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	United States	2021-09-25	2020	PG-13	90 min
1	s3	TV Show	Ganglands	Julien Leclercq	France	2021-09-24	2021	TV-MA	1 Season
2	s6	TV Show	Midnight Mass	Mike Flanagan	United States	2021-09-24	2021	TV-MA	1 Season
3	s14	Movie	Confessions of an Invisible Girl	Bruno Garotti	Brazil	2021-09-22	2021	TV-PG	91 min
4	s8	Movie	Sankofa	Haile Gerima	United States	2021-09-24	1993	TV-MA	125 min

In [5]:
```
# Convert date_added into datetime format
df["date_added"] = pd.to_datetime(
    df["date_added"],
    errors="coerce"
)

# Remove rows where the date is missing
trend_df = df.dropna(subset=["date_added"]).copy()

# Extract the year
trend_df["year_added"] = trend_df["date_added"].dt.year
```

```
# Count titles added each year
yearly_data = (
    trend_df.groupby("year_added")
    .size()
    .reset_index(name="title_count")
)

print("Yearly Netflix content additions:")
display(yearly_data)
```

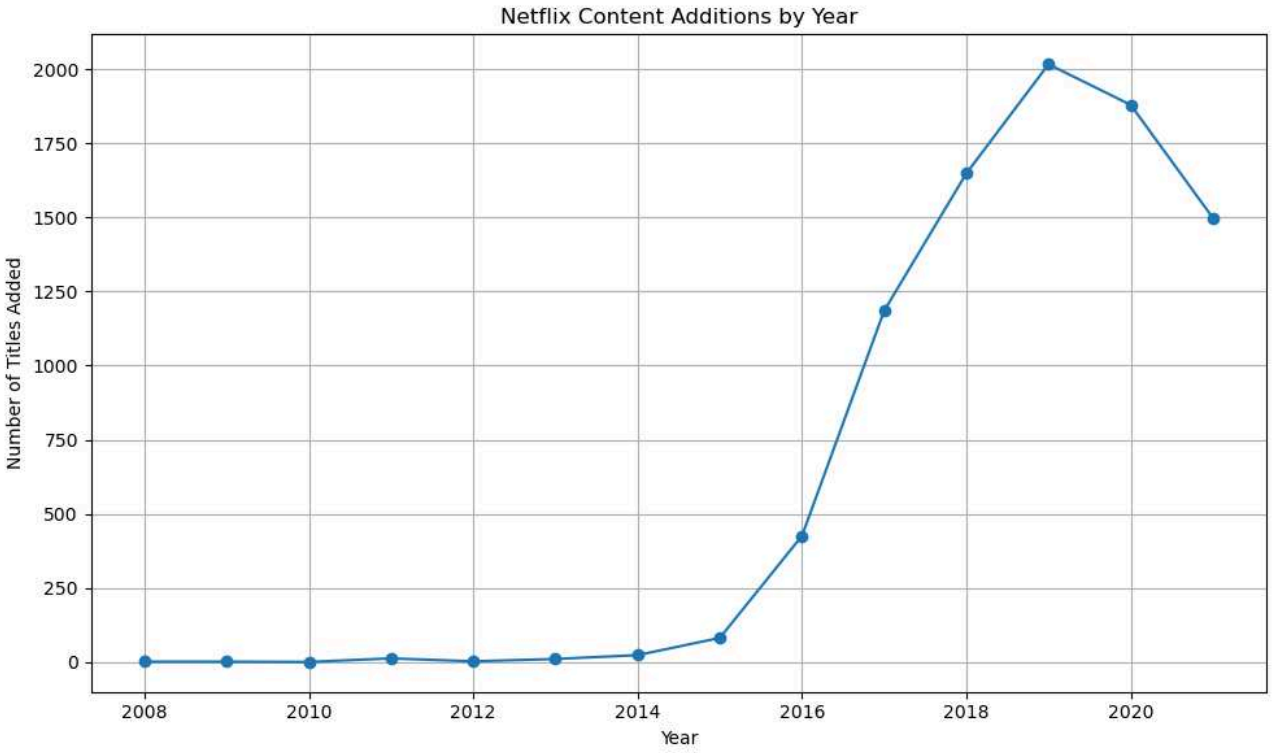
```
Yearly Netflix content additions:
```

	year_added	title_count
0	2008	2
1	2009	2
2	2010	1
3	2011	13
4	2012	3
5	2013	11
6	2014	24
7	2015	82
8	2016	426
9	2017	1185
10	2018	1648
11	2019	2016
12	2020	1879
13	2021	1498

In [6]:
```
plt.figure(figsize=(10, 6))

plt.plot(
    yearly_data["year_added"],
    yearly_data["title_count"],
    marker="o"
)
plt.title("Netflix Content Additions by Year")
plt.xlabel("Year")
plt.ylabel("Number of Titles Added")

plt.grid(True)
plt.tight_layout()
plt.show()
```



In [7]:
```python
# Prepare input and output variables
X = yearly_data[["year_added"]]
y = yearly_data["title_count"]

# Train the model
model = LinearRegression()
model.fit(X, y)

print("Linear Regression model trained successfully!")
```

```
Linear Regression model trained successfully!
```

In [8]:
```python
# Identify the last year in the dataset
last_year = int(yearly_data["year_added"].max())

# Create the next three years
future_years = pd.DataFrame({
    "year_added": [last_year + 1, last_year + 2, last_year + 3]
})

# Predict title additions
future_years["predicted_titles"] = model.predict(
    future_years[["year_added"]]
)

# Round predictions to whole numbers
future_years["predicted_titles"] = (
    future_years["predicted_titles"]
    .round()
    .astype(int)
```

```
    .clip(lower=0)
)

print("Predicted Netflix Content Additions:")
display(future_years)
```

Predicted Netflix Content Additions:

	year_added	predicted_titles
0	2022	1895
1	2023	2064
2	2024	2233

In [9]:
```
# Predict historical values
historical_predictions = model.predict(X)

mae = mean_absolute_error(y, historical_predictions)
r2 = r2_score(y, historical_predictions)

print("Model Evaluation")
print("----------------")
print("Mean Absolute Error:", round(mae, 2))
print("R-squared Score:", round(r2, 3))
```

```
Model Evaluation
----------------
Mean Absolute Error: 346.07
R-squared Score: 0.753
```

In [10…
```
plt.figure(figsize=(10, 6))

# Historical data
plt.plot(
    yearly_data["year_added"],
    yearly_data["title_count"],
    marker="o",
    label="Historical Data"
)

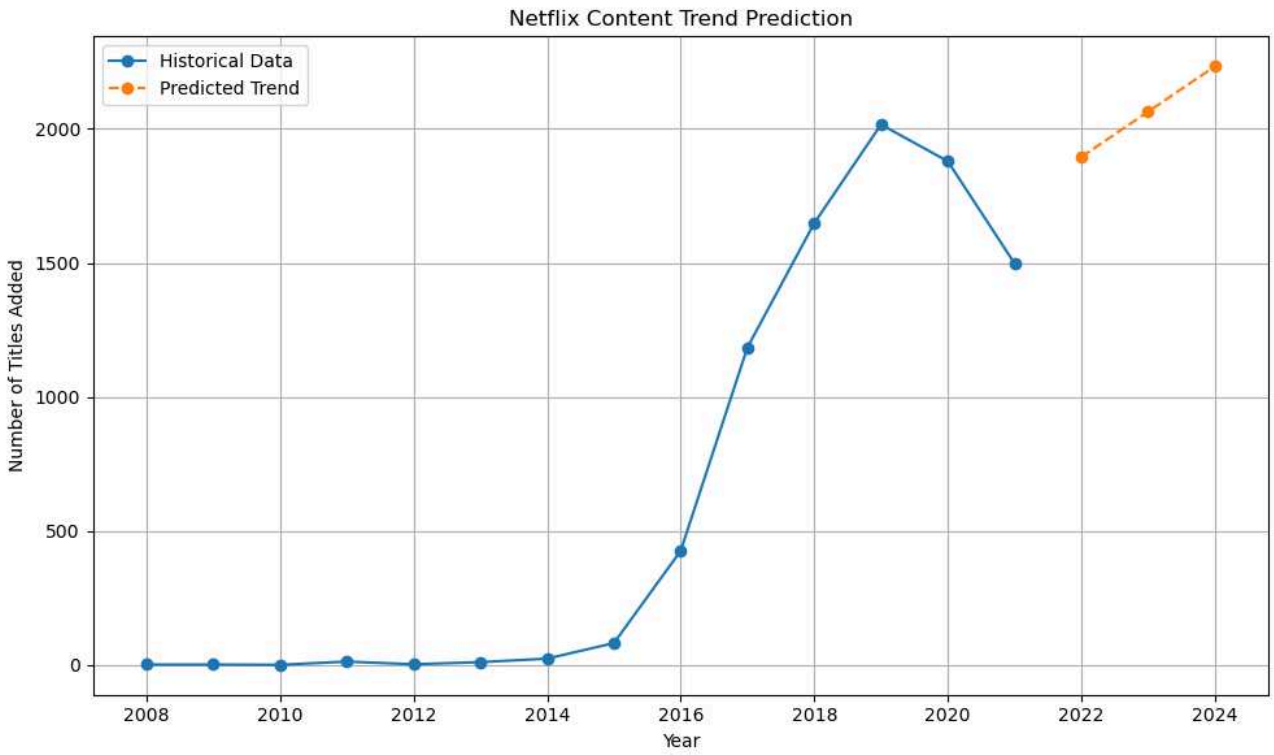
# Future predictions
plt.plot(
    future_years["year_added"],
    future_years["predicted_titles"],
    marker="o",
    linestyle="--",
    label="Predicted Trend"
)

plt.title("Netflix Content Trend Prediction")
plt.xlabel("Year")
plt.ylabel("Number of Titles Added")

plt.legend()
```

```
plt.grid(True)
plt.tight_layout()

plt.show()
```



In [11…

```
import os

os.makedirs("Graphs", exist_ok=True)

plt.figure(figsize=(10, 6))

plt.plot(
    yearly_data["year_added"],
    yearly_data["title_count"],
    marker="o",
    label="Historical Data"
)

plt.plot(
    future_years["year_added"],
    future_years["predicted_titles"],
    marker="o",
    linestyle="--",
    label="Predicted Trend"
)

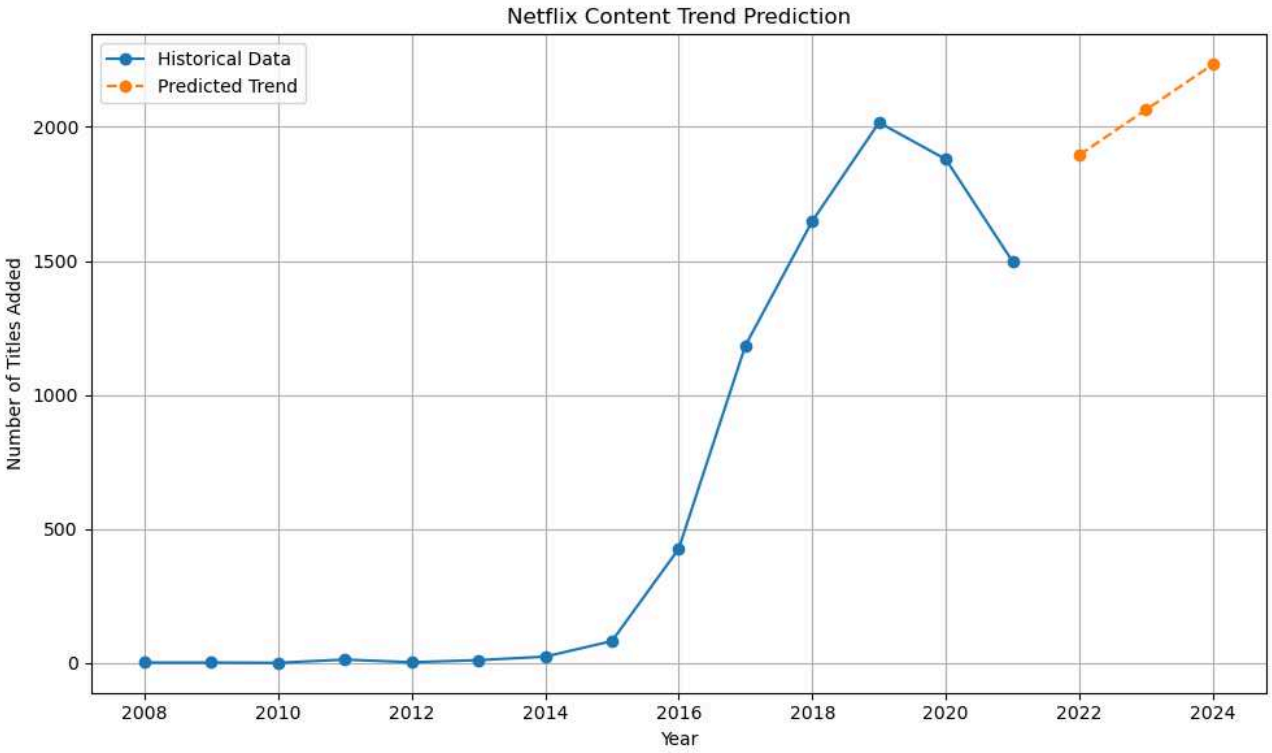
plt.title("Netflix Content Trend Prediction")
plt.xlabel("Year")
plt.ylabel("Number of Titles Added")
```

```
plt.legend()
plt.grid(True)
plt.tight_layout()

plt.savefig(
    "Graphs/netflix_trend_prediction.png",
    bbox_inches="tight"
)

plt.show()

print("Trend prediction graph saved successfully!")
```




```
Trend prediction graph saved successfully!
```

Trend Prediction Analysis — Conclusion

Objective

To analyze historical Netflix content additions and estimate future trends using Linear Regression.

Methodology

1. Loaded the cleaned Netflix dataset.
2. Converted the date-added column into a datetime format.
3. Counted titles added in each year.
4. Visualized historical content additions.
5. Trained a Linear Regression model.
6. Generated estimates for the next three years.
7. Evaluated the model using MAE and R-squared.

Conclusion

The analysis demonstrates how historical data can be used to identify trends and generate preliminary forecasts. Linear Regression provides a simple baseline for estimating future Netflix content additions. However, these estimates depend on historical patterns and should not be interpreted as guaranteed future values.